WealthWise: Requirements and .NET Component Guide

Nov 4, 2026 · @Biju Tholath

About this document

This guide explains what WealthWise is for, what it must do, and what each .NET piece of it does, in everyday language a non-programmer can follow.

WealthWise helps a financial advisor look after clients' investment accounts. It shows what each account is worth, whether it has wandered away from the plan agreed with the client, suggests trades to correct that, lets the advisor place trades, and keeps a record of what happened.

If you are...	Read
A business owner or analyst	Sections 2 to 4 (the business, and what the system must do)
A tester or technical lead	Sections 4 to 6 and 10 (requirements, structure, traceability)
A new developer	Sections 6 to 9 and the glossary in section 11

Requirement numbers use three prefixes: **BR** for business requirements, **FR** for functional requirements (what the system does) and **TR** for technical requirements (how well and with what it does it). Section 10 links each one to the part of the code that satisfies it.

A companion document, *WealthWise: What the Web App and API Do*, walks through every screen and every API call. The prices and securities in the demo are simulated and are not investment advice.

The business in plain English

An advisor's job is to keep each client's money matching an agreed plan, and WealthWise automates the checking, the arithmetic and the record-keeping that job needs.

The problem. A client's plan might say: 50% in shares, 35% in bonds, 5% in cash, 10% in property and gold. That split is the **target**. Markets move, so a few months later the account may hold 59% in shares. That gap is called **drift**. Checking drift by hand across many accounts in spreadsheets is slow, easy to get wrong, and leaves no trustworthy history.

What the system does, in four verbs.

1. **See** what each account is worth and how it is split.
2. **Compare** that split with the plan and flag accounts that have drifted.
3. **Act** by suggesting corrective trades and letting the advisor place trades.
4. **Record** every trade so it can be reviewed later.

Who uses it.

Person	What they do with it
Advisor	The everyday user: reviews accounts, previews rebalancing, places trades
Client	Does not log in; their plan is what the system measures against
Reviewer or auditor	Reads the activity trail to check what was done and when
Platform team	Runs, monitors and updates the system

A restaurant analogy makes the pieces easy to remember:

In a restaurant	In WealthWise
Dining room: menu and waiter	The web app the advisor sees
Host at the door who checks reservations and limits the crowd	The gateway (Kong): checks the key, limits requests
The kitchen, where orders are prepared	The API, the .NET program
The kitchen's house rules (never serve what you do not have)	The Domain project: business rules
Pantry and filing cabinet	The PostgreSQL database

In a restaurant	In WealthWise
The ticket rail that passes jobs along for later	The message queue
The bookkeeper who writes every order in a ledger	The event consumer and the audit log

Business requirements

The business needs ten things from WealthWise, and each one answers a concrete worry an advisory firm has.

ID	The business needs to…	Why it matters
BR-1	See every client account in one place: value, split and gain	Saves hours of spreadsheet work and shows who needs attention first
BR-2	Detect when an account has drifted from the client's agreed plan	Keeping to the plan is a core duty to the client
BR-3	Get suggested corrective trades, and preview the result before acting	Advisors decide faster and avoid guesswork
BR-4	Place trades safely: never spend money that is not there, never sell what is not owned	Prevents costly mistakes at the source
BR-5	Keep a trustworthy record of every trade	Needed for review, complaints and audits
BR-6	Never lose or double-count a record, even under heavy or repeated activity	A ledger that cannot be trusted is worthless
BR-7	Allow only authorised use and limit abuse	Client information must be protected
BR-8	Run on an ordinary laptop with free tools	Low cost to build, demonstrate and learn from
BR-9	Change easily: new rebalancing policies, a real price feed, a bigger scale	Business rules change faster than software should have to
BR-10	Be built and released in small, reviewed, visible steps	The team and stakeholders can see progress and trust each release

Deliberately out of scope for this version:

- Moving real money or connecting to a real broker or exchange
- Client log-in and individual advisor accounts (access is by a shared key)
- Tax rules, fees and regulatory reporting
- Real market prices (prices are simulated)

Functional requirements

The system must do twenty-five specific things, and each has a plain test for whether it works. "Serves" points back to the business requirement it satisfies.

ID	Area	The system must…	You know it works when…	Serves
FR-1	Portfolios	List all accounts with client, risk profile, value, unrealized gain and largest drift	The three demo accounts appear, sorted by name; drift above 5 points is flagged	BR-1, BR-2
FR-2	Portfolios	Show one account: total value, cash, cost basis, gain and every holding	Holdings are listed largest first with quantity, price, value and gain	BR-1
FR-3	Portfolios	Show each asset class's actual share, target share and drift	Shares of Equities, Fixed income, Cash and Alternatives add up to 100%	BR-2
FR-4	Portfolios	Re-price the open account automatically	Values change every 15 seconds without reloading the page	BR-1
FR-5	Valuation	Value a holding as quantity times current price, rounded to cents	A figure checks out with a calculator	BR-1
FR-6	Valuation	Track what was paid (cost basis) using the average-cost method	Selling 4 of 10 shares removes 40% of the cost paid	BR-1, BR-5
FR-7	Trading	Execute a buy or sell at the current market price	The API answers 201 Created; cash, holding and trade history all update	BR-4
FR-8	Trading	Reject a buy that costs more than the cash available	HTTP 422 with a message stating cost and cash	BR-4
FR-9	Trading	Reject a sell larger than the position held	HTTP 422 naming the position held	BR-4

ID	Area	The system must…	You know it works when…	Serves
FR-10	Trading	Reject zero or negative quantities and unknown symbols	HTTP 422 with a clear message	BR-4
FR-11	Trading	Stop two simultaneous trades from both succeeding on stale data	The second gets HTTP 409 and is told to reload	BR-4, BR-6
FR-12	Trading	Show recent trades for an account	The page shows the latest 15; the API can return up to 200	BR-5
FR-13	Rebalancing	Offer a threshold plan that only trades classes outside a band (default 5 points)	An account sitting exactly 5.00 points off is not traded at a 5-point band	BR-3
FR-14	Rebalancing	Offer a full plan that returns every non-cash class to target, ignoring trades under $50	The proposed trades bring each class to its target share	BR-3
FR-15	Rebalancing	Treat cash as the funding source, and report cash after trading and the projected split	The response lists cash after and projected drift per class	BR-3
FR-16	Rebalancing	Reject an unknown strategy name	HTTP 422 saying use threshold or full	BR-3
FR-17	Audit	Announce each executed trade as a trade.executed event on the queue	The event arrives after the trade is saved	BR-5
FR-18	Audit	Record each event once, even if it is delivered twice	Five trades make exactly five audit rows	BR-5, BR-6
FR-19	Audit	Show recent activity, newest first	The Activity page lists up to 40 entries	BR-5
FR-20	Gateway	Refuse calls without a valid key	HTTP 401	BR-7
FR-21	Gateway	Refuse more than 120 requests in a minute	HTTP 429	BR-7

ID	Area	The system must…	You know it works when…	Serves
FR-22	Gateway	Stamp every response with a correlation ID	The X-Correlation-ID header is present	BR-7, BR-10
FR-23	Platform	Report whether the service is alive	The health address answers 200	BR-10
FR-24	Platform	Create demo data on first start	Three accounts and seven securities exist	BR-8
FR-25	Platform	Report problems in one standard format with a readable message	Errors arrive as problem-details with a title and detail	BR-4, BR-10

Technical requirements

The technical requirements say how the system is built and how well it must behave; each has a plain-language translation.

ID	Area	The system must…	In plain terms	Serves
TR-1	Structure	Split the .NET code into four projects (Domain, Application, Infrastructure, Api) whose dependencies point only inward	Business rules never depend on the database or the web, so each can change alone	BR-9
TR-2	Platform	Use .NET 10, ASP.NET Core minimal APIs, C# with null-safety checks on	Current, supported tools, with the compiler catching missing-value mistakes	BR-8
TR-3	Data	Store data in PostgreSQL through Entity Framework Core, with exact decimals for money (cash to 2 places, prices to 4), unique holding per account and symbol, and enforced links between tables	No rounding drift; the database itself blocks nonsense data	BR-6

ID	Area	The system must…	In plain terms	Serves
TR-4	Data	Create the database tables itself at start-up and keep retrying for about a minute while the database boots	The app starts cleanly whichever container wakes first	BR-8
TR-5	Data	Carry a version number on each account that changes with every trade	Two people editing the same account cannot silently overwrite each other	BR-6
TR-6	Messaging	Use an SQS-compatible queue; assume messages can arrive more than once; record each by a unique event ID	Like a post office that occasionally delivers a letter twice, the ledger ignores the second copy	BR-5, BR-6
TR-7	Messaging	Publish the event only after the trade is saved; a failed publish is logged but never undoes the trade	The money movement is never rolled back because a notice failed	BR-4, BR-5
TR-8	Prices	Fetch prices through a replaceable price provider, simulated for now and cached for 5 seconds	A real market feed can replace the fake one later; repeated screens show steady prices	BR-9
TR-9	Interface	Speak JSON with camelCase names and enums written as words	Easy for any program or person to read	BR-9
TR-10	Interface	Publish a machine-readable OpenAPI description at /openapi/v1.json	Other teams can learn the API without asking	BR-9, BR-10
TR-11	Errors	Return standard problem-details responses; log expected rejections (404, 422, 409) quietly and unexpected faults as errors	Logs show real problems, not routine rejections	BR-10

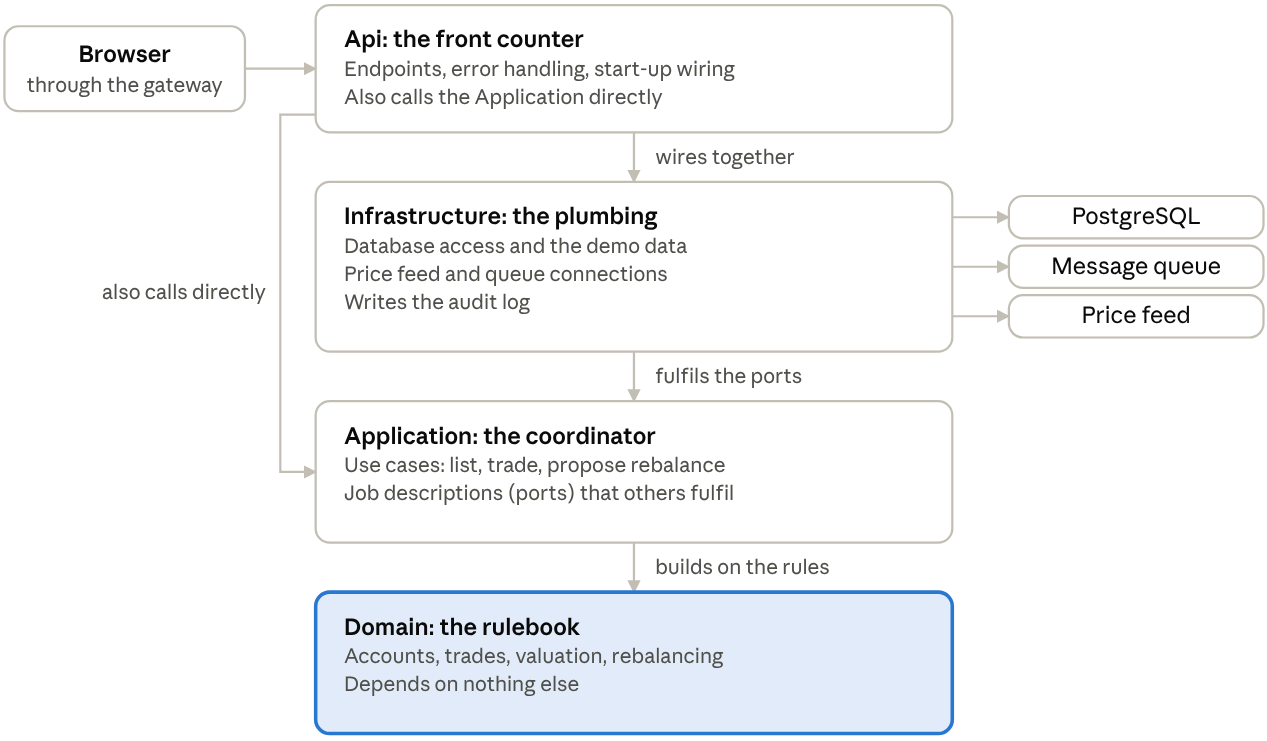
ID	Area	The system must…	In plain terms	Serves
TR-12	Security	Put a gateway (Kong) in front with key check, 120-per-minute limit, cross-origin rules and correlation IDs; inject the key on the server so it never ships to the browser	A guarded front door with a visitor log number on every request	BR-7
TR-13	Operations	Package as container images that run on Docker Compose and on Kubernetes, with health checks, resource limits, a non-root user and a read-only file system for the API	Same software, safer defaults, same behaviour on a laptop or a cluster	BR-8, BR-10
TR-14	Operations	Expose /health and /api/health for automatic monitoring	Platforms can tell if the service is alive	BR-10
TR-15	Operations	Take all settings (database, queue, ports) from environment variables or settings files	Change environments without rebuilding	BR-8
TR-16	Resilience	Retry the queue every 5 seconds after a failure and let the platform restart a crashed container	Short outages heal themselves	BR-6
TR-17	Quality	Cover the rules with 10 unit tests and the API with 5 integration tests that run without Docker	Mistakes in the rules are caught in seconds	BR-10
TR-18	Delivery	Build, test and smoke-test every change in GitHub Actions; lint scripts; release images and hand them to Octopus Deploy	Every change is checked the same way before it is trusted	BR-10

ID	Area	The system must…	In plain terms	Serves
TR-19	Time	Store and compare all times in UTC	No daylight-saving surprises	BR-5
TR-20	Performance	Serve screens from in-process calculations with no formal speed target; provide a load script that reports latency	Fast enough for a demo; measurable if that changes	BR-8

The big picture

The four .NET projects stack like layers of a cake, and each layer may only look downward, never up.

Every arrow points inward: the Domain depends on nothing



Arrows point from a part to what it depends on. The Domain depends on nothing.

the four .NET projects and what each depends on

Read it top to bottom: the Api wires everything together, Infrastructure fulfils the job descriptions that Application writes, and Application builds on the rules in the Domain. The Api also calls Application directly (the left-hand line) to run the use cases.

Why the rule matters. The rulebook can be tested without any database, and the database, price feed or queue can be swapped without touching a single business rule. That is how BR-9 (easy to change) is met.

What lives outside the .NET code: the Angular web app, the Kong gateway, PostgreSQL, the queue, and the container and Kubernetes setup. The companion document covers the first of these.

Component guide: the Domain project

The Domain project is the rulebook: it holds the business ideas (accounts, holdings, trades) and the arithmetic, and it knows nothing about databases, websites or queues. That isolation is why its ten unit tests run in milliseconds.

Piece	What it is, in plain words	What it does and enforces
AssetClass, TradeSide, RiskProfile	Three fixed pick-lists	Asset classes: Equity, FixedIncome, Cash, Alternatives. Sides: Buy, Sell. Risk profiles: Conservative, Balanced, Growth
DomainException	The "you cannot do that" signal	Thrown whenever a business rule is broken; the API turns it into an HTTP 422
Security	One thing you can buy (an ETF)	Symbol (always upper-case), name, asset class and a base price that must be positive

Piece	What it is, in plain words	What it does and enforces
Holding	How many units of one security an account owns, and what was paid in total	Adds units and cost on a buy; removes units and a proportional share of cost on a sell (average-cost method)
Trade	One line in the account's history	Side, quantity, price, time; its notional value is quantity times price rounded to cents
TargetAllocation	The client's plan for one asset class	A weight between 0 and 1, for example 0.50 for 50%
Portfolio	The account itself, and the only door through which cash and holdings change	Requires a name and non-negative opening cash; its targets must add up to 100%; runs every trade through ApplyTrade; carries a version number that rises with each trade
Valuator	A calculator that prices an account	Turns holdings and prices into total value, gain and the actual-versus-target table (no database, no side effects)
FullRebalanceStrategy, ThresholdRebalanceStrategy	Two rebalancing policies	Full returns every non-cash class to target (ignoring trades under $50); Threshold only touches classes outside a tolerance band

Piece	What it is, in plain words	What it does and enforces
RebalanceStrategyFactory	The policy picker	Turns the word "threshold" or "full" into the right policy; blank means threshold; anything else is rejected; a band outside 0 to 1 falls back to 5%

How a trade is applied (Portfolio.ApplyTrade), step by step.

1. Refuse a quantity or price that is zero or negative.
2. Work out the cost: quantity times price, rounded to cents.
3. For a **buy**: refuse if the cost is more than the cash; otherwise take the cash and add the shares and their cost to the holding (creating it if new).
4. For a **sell**: refuse if the account holds fewer shares than requested; otherwise add the cash, remove the shares and a matching share of the cost, and delete the holding if nothing is left.
5. Raise the account's version number by one.
6. Add a trade record to the history.

Worked example. An account has $10,000. It buys 10 shares at $100: cash falls to $9,000 and the holding is 10 shares costing $1,000. It then sells 4 at $120: cash rises by $480 to $9,480, and the holding is 6 shares costing $600 (the cost of the 4 sold, $400, left with them). The version number is now 2. This exact scenario is one of the unit tests.

Component guide: Application, Infrastructure, Api and tests

The other three projects wrap the rulebook: Application decides what happens in what order, Infrastructure talks to the outside world, and Api is the front counter.

Application project: the coordinator

It contains the use cases and the "ports", which are job descriptions such as "someone must be able to fetch prices" without saying who. That is what lets the database or price feed be swapped without touching the logic.

Piece	In plain words	What it does
IPortfolioRepository	The filing clerk's job description	Load or save accounts, trades and securities
IPriceProvider	The price-board job description	Return the current price for a list of securities
IEventPublisher	The messenger's job description	Announce that something happened
IAuditLog	The ledger keeper's job description	Record an event once; list recent events
PortfolioService	Answers questions about accounts	List accounts with value and largest drift; show one account; propose a rebalance and the projected result; list trades
TradeService	Carries out a trade from start to finish	See the steps below
DTOs (PortfolioSummary, PortfolioDetail, RebalancePlan, TradeRequest, TradeDto, SecurityDto)	The shapes of what travels in and out	Plain bundles of fields for the screen and the API
NotFoundException	The "no such account" signal	Becomes HTTP 404

What TradeService does for one order.

1. Load the account (404 if it does not exist).
2. Tidy the symbol (trim, upper-case) and confirm the security exists.
3. Ask the price provider for the current price.
4. Hand the order to the account's ApplyTrade, which enforces the rules.
5. Save the changes to the database.
6. Announce "trade.executed"; if that fails, log a warning but keep the trade.

Infrastructure project: the plumbing

It talks to PostgreSQL, the price source and the queue.

Piece	In plain words	Detail
WealthWiseDbContext	The map between code and database tables	Defines tables, precision of money columns, links between tables, a unique holding per account and symbol, and the version check
EfPortfolioRepository	The filing clerk	Loads accounts with their holdings and targets; lists trades newest first
EfAuditLog	The ledger keeper	Refuses to store the same event ID twice, even if two writers race
DbInitializer	The opening-day crew	Waits for the database (up to 30 tries, 2 seconds apart), builds the tables and, on first run, adds 7 securities and 3 demo accounts
SimulatedPriceProvider	A pretend market	Base price plus gentle waves; swings are about 3% for equities, 3.5% for alternatives, 1.2% for bonds, 0.04% for cash
CachingPriceProvider	A short memory	Reuses a price for 5 seconds so a screen and a trade agree
SqsEventPublisher	Posts a letter	Wraps the event with an ID, type and time, and sends it to the queue
NullEventPublisher	A mailbox that is switched off	Used when no queue is configured (tests, lite mode)
TradeEventConsumer	The post-room clerk	Checks the queue (waiting up to 5 seconds, taking up to 10 messages), records each in the audit log, then removes the message; on trouble it waits 5 seconds and tries again

Piece	In plain words	Detail
AddInfrastructure	The wiring diagram	Chooses PostgreSQL or an in-memory database (for tests), turns the queue on or off

Api project: the front counter

Program.cs is the whole program: it assembles the pieces, sets up error handling, and defines the endpoints (listed in the companion document).

- **Two layers of error handling.** An inner layer turns expected problems (no such account, a broken rule, a stale version) into a polite response and a quiet log line. An outer layer catches genuine bugs, answers HTTP 500 and logs them as errors.
- **Enums as words.** Values such as Buy or Equity travel as text.
- **OpenAPI.** A machine-readable description is served at /openapi/v1.json.
- **Health.** /health and /api/health say the service is alive.
- **Container.** The Dockerfile builds the program and runs it as a non-root user on port 8080.

Test project: the safety net

Group	Count	What it proves
DomainTests	10	Buys and sells move cash correctly; bad orders are refused; cost basis is proportional; targets must total 100%; allocation adds to 100%; both rebalance policies behave; the factory rejects unknown names
ApiTests	5	Three demo accounts load; unknown accounts give 404; an oversized buy gives 422; a trade moves cash and appears in history; the rebalance endpoint works and rejects a bad strategy

Design patterns, explained simply

A design pattern is a proven way to solve a common construction problem, and WealthWise uses nine of them.

Pattern	Everyday picture	Where it lives	What it buys
Aggregate root	A bank account that only the teller may change	Portfolio and its ApplyTrade	The rules cannot be bypassed; cash and holdings never disagree
Strategy	Choosing walking, cycling or driving on a map app	IRebalanceStrategy with the Full and Threshold policies	A new rebalancing policy is a new class, not a rewrite
Factory	A vending machine: press a word, get the right item	RebalanceStrategyFactory	Callers ask by name and never build policies themselves
Decorator	Gift-wrap that adds something without changing the gift	CachingPriceProvider around SimulatedPriceProvider	Caching added with no change to the price code
Ports and adapters (repository)	A wall socket: any plug that fits works	IPortfolioRepository, IPriceProvider, IEventPublisher, IAuditLog	Swap the database, feed or queue freely
Dependency injection	A stage manager handing each actor the right props	AddApplication and AddInfrastructure	Parts are assembled in one place and are easy to replace in tests
Pure function	A pocket calculator: same input, same answer, no side effects	Valuator.Calculate	Trivial to test and to trust
Idempotent consumer	Signing for a parcel once, even if the courier rings twice	EfAuditLog with a unique event ID	Repeated delivery cannot double-count

Pattern	Everyday picture	Where it lives	What it buys
Optimistic concurrency	Two people editing one shared page: the second save is told the page changed	Portfolio.Version	No silent overwrites without slowing everyone down with locks

Traceability: from requirement to code to proof

Every business requirement is met by named code and checked by a named test or script, so nothing in the list is only a promise.

Business requirement	Supporting FR and TR	Built in	Checked by
BR-1 See every account	FR-1 to 6, TR-9	PortfolioService, Valuator, the web pages	ApiTests (seeded accounts), DomainTests (valuation), smoke step 3
BR-2 Detect drift	FR-1, 3, 13	Valuator allocation lines	DomainTests (allocation sums to 100%)
BR-3 Suggest and preview trades	FR-13 to 16	Rebalancing.cs, PortfolioService.ProposeRebalanceAsync	DomainTests (full, threshold, factory), ApiTests (rebalance)
BR-4 Trade safely	FR-7 to 11, TR-5, TR-7	Portfolio.ApplyTrade, TradeService, error handling in Program.cs	DomainTests (buy, sell, refusals), ApiTests (422), smoke step 5

Business requirement	Supporting FR and TR	Built in	Checked by
BR-5 Keep a record	FR-12, 17 to 19, TR-6, TR-19	SqsEventPublisher, TradeEventConsumer, EfAuditLog	Smoke step 6, the idempotency check
BR-6 No loss or double-counting	FR-11, 18, TR-3, 5, 6, 16	Database constraints, Portfolio.Version, unique event ID	The idempotency check (5 trades, 5 rows)
BR-7 Authorised use only	FR-20 to 22, TR-12	Kong configuration, nginx key injection	Smoke step 1 (401 without a key)
BR-8 Runs on a laptop	FR-24, TR-2, 4, 13, 15	DbInitializer, Docker Compose, Kubernetes manifests, lite mode	The full verification run
BR-9 Easy to change	TR-1, 8, 9, 10	The four-project layering, ports, strategies, OpenAPI	Review of dependencies; the OpenAPI file
BR-10 Small, visible steps	FR-22, 23, 25, TR-11, 13, 14, 17, 18	GitHub Actions, health endpoints, Jira and Confluence material	The CI pipeline on every change

Known limitations, next steps and glossary

WealthWise is a teaching-grade system, and these limits are deliberate or known, not hidden.

Known limitations

- Access is by one shared key, with no individual log-in or roles.
- Prices are simulated and the securities are generic ETFs; everything is in one currency.
- Rebalancing proposes amounts per asset class, not specific securities, and ignores tax and fees.

- If the program stops between saving a trade and announcing it, the audit entry for that trade is lost. A transactional outbox is the planned fix.
- The database tables are created automatically rather than by versioned migrations, which should change before a real deployment.
- The API reads at most 200 audit rows per call.

Planned next steps (all in the backlog): execute a whole rebalance as a basket of orders, tax-aware lot selection, proper log-in through an identity provider, front-end tests, deployment to AWS containers, and a serverless version of the audit consumer.

Glossary

Term	Plain meaning
Portfolio	One client's investment account
Security	Something you can buy or sell, such as a fund
Holding	How much of one security an account owns
Asset class	A family of investments: shares (Equity), bonds (FixedIncome), Cash, or Alternatives such as property and gold
Target allocation	The share of the account the plan wants in each asset class
Drift	Actual share minus target share
Band	How much drift is tolerated before action is suggested
Rebalance	Trading to bring the account back toward its targets
Cost basis	Total paid for what is still held
Unrealized gain	Today's value minus cost basis, before anything is sold
Notional	The total money value of a trade
API	The program other programs and screens talk to
Endpoint	One address on the API that does one job

Term	Plain meaning
Gateway	The guarded front door that checks keys and limits traffic
Queue	A line of messages waiting to be handled
Event	A message saying that something happened
Audit log	The permanent record of events
Idempotent	Safe to repeat: doing it twice has the same effect as once
Container	A sealed package of a program and everything it needs to run
Kubernetes	Software that runs and supervises many containers
Ingress	The single entrance to the Kubernetes cluster